

TP1-ASSR

Thibault Malespine

25 septembre 2026

## Table des matières

|          |                                                 |          |
|----------|-------------------------------------------------|----------|
| <b>I</b> | <b>Mise en place d'un tunnel IPSec sans IKE</b> | <b>3</b> |
| I        | Architecture Réseaux . . . . .                  | 3        |
| II       | Ajout des SA . . . . .                          | 3        |
| III      | Ajout des SP . . . . .                          | 4        |
| IV       | Analyse du trafic en mode tunnel . . . . .      | 4        |
| V        | Analyse du trafic en mode transport . . . . .   | 4        |

# I Mise en place d'un tunnel IPSec sans IKE

## I Architecture Réseaux

Pour la mise en place de l'architecture réseaux de ce TP, nous avons réutilisé le script bash `build_architecture` fourni en annexe qui s'occupe de construire le réseau et de faire le routage des machines.

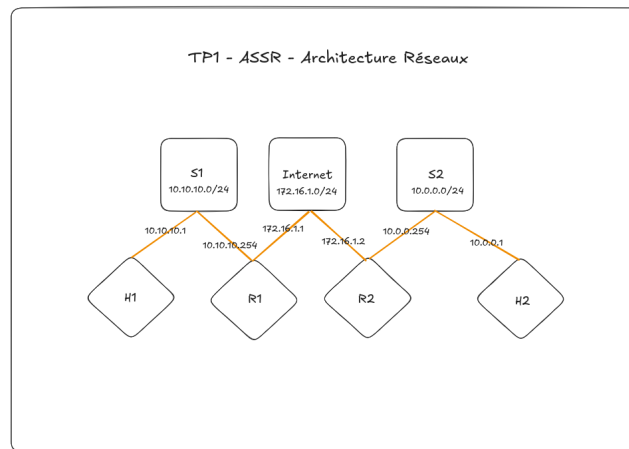


FIGURE 1 – Architecture réseaux du TP

## II Ajout des SA

Pour créer un tunnel IPSec, la première chose à faire est de créer des associations de sécurité qui vont définir le protocole et les algorithmes utilisés pour assurer une communication entre deux machines.

```
ip xfrm state add src 10.10.10.1 dst 10.0.0.1 proto esp spi "$SPI1"
reqid "$Rid1" mode tunnel auth sha256 "$K1" enc aes "$K2"
```

```
ip xfrm state add src 10.0.0.1 dst 10.10.10.1 proto esp spi "$SPI2"
reqid "$Rid2" mode tunnel auth sha256 "$K3" enc aes "$K4"
```

On utilise des clés en dur pour l'algorithme d'authentification (sha256) et pour celui de chiffrement (aes) utilisé par le protocole ESP.

On a besoin de 4 clés au total, 2 pour les échanges allant de h1 vers h2 et 2 pour les échanges allant de h2 vers h1.

On utilise également des identifiants pour le SPI qui sert à identifier la SA dans la SADB, et un identifiant (reqid) qui permet d'associer la SA à une SP.

Ces SAs sont déclarées de la même manière sur les deux machines.

### III Ajout des SP

Maintenant il faut définir les SP sur chacune des machines pour identifier les paquets qui vont transiter par le tunnel.

Sur H1 on exécute

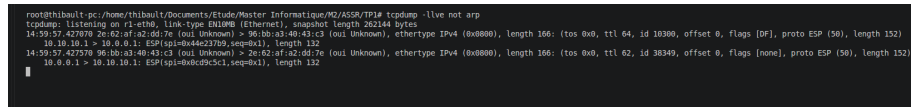
```
ip xfrm policy add src 10.0.0.1 dst 10.10.10.1 dir in tmpl src 10.0.0.1
dst 10.10.10.1 proto esp reqid "$Rid2" mode tunnel
ip xfrm policy add src 10.10.10.1 dst 10.0.0.1 dir out tmpl src
10.10.10.1 dst 10.0.0.1 proto esp reqid "$Rid1" mode tunnel
```

Et sur H2 on exécute

```
ip xfrm policy add src 10.10.10.1 dst 10.0.0.1 dir in tmpl src 10.10.10.1
dst 10.0.0.1 proto esp reqid "$Rid1" mode tunnel
ip xfrm policy add src 10.0.0.1 dst 10.10.10.1 dir out tmpl src
10.0.0.1 dst 10.10.10.1 proto esp reqid "$Rid2" mode tunnel
```

### IV Analyse du trafic en mode tunnel

Regardons ce que produit l'envoi d'un ping de h1 vers h2 avec notre configuration actuelle.

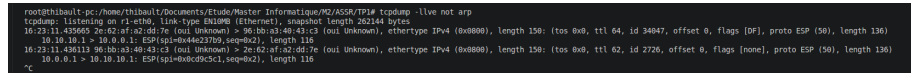


```
root@thibault-pc:~/home/thibault/Documents/Etude/Master Informatique/M2/ASIP/TP1# tcpdump -l -i eth0 not arp
tcpdump: listening on r1-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
14:59:57.427070 26:02:af:a2:dd:7e (oui Unknown) > 96:1b:a3:48:43:c3 (oui Unknown), ethertype IPv4 (0x0800), length 166: (tos 0x0, ttl 64, id 10300, offset 0, flags [DF], proto ESP (50), length 152)
  10.10.10.1 > 10.0.0.1: ESP(spi=0x44e237b9,seq=0x1), length 152
14:59:57.427570 96:1b:a3:48:43:c3 (oui Unknown) > 26:02:af:a2:dd:7e (oui Unknown), ethertype IPv4 (0x0800), length 166: (tos 0x0, ttl 62, id 38349, offset 0, flags [none], proto ESP (50), length 152)
  10.0.0.1 > 10.10.10.1: ESP(spi=0x44e237b9,seq=0x1), length 152
```

FIGURE 2 – Dump d'un ping de h1 vers h2 sur r1 en mode tunnel

On voit sur la capture d'écran que le message ICMP a été encapsulé dans un paquet ESP, ce qui correspond à l'utilisation d'IPSec.

### V Analyse du trafic en mode transport



```
root@thibault-pc:~/home/thibault/Documents/Etude/Master Informatique/M2/ASIP/TP1# tcpdump -l -i eth0 not arp
tcpdump: listening on r1-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
16:23:11.43565 26:02:af:a2:dd:7e (oui Unknown) > 96:1b:a3:48:43:c3 (oui Unknown), ethertype IPv4 (0x0800), length 150: (tos 0x0, ttl 64, id 34047, offset 0, flags [DF], proto ESP (50), length 136)
  10.10.10.1 > 10.0.0.1: ESP(spi=0x44e237b9,seq=0x2), length 136
16:23:11.436113 96:1b:a3:48:43:c3 (oui Unknown) > 26:02:af:a2:dd:7e (oui Unknown), ethertype IPv4 (0x0800), length 150: (tos 0x0, ttl 62, id 2726, offset 0, flags [none], proto ESP (50), length 136)
  10.0.0.1 > 10.10.10.1: ESP(spi=0x44e237b9,seq=0x2), length 136
```

FIGURE 3 – Dump d'un ping de h1 vers h2 sur r1 en mode transport

On voit que la taille de la trame Ethernet est plus grande en mode tunnel par rapport au mode transport (166 octets contre 150 octets), cela est attendu puisqu'en mode transport seule la charge utile est chiffrée et l'en-tête IP d'origine du paquet est conservé, alors qu'en mode tunnel le paquet IP entier est chiffré et un nouvel en-tête IP est ajouté par-dessus.